

3/display.asm

```
1 ; ClearDisplay
2 ;
3 ; Description:      Clears all LEDs on the pinball machine (7-segments and lights).
4 ;
5 ; Operation:       Iterates through the display RAM buffers and writes zeros
6 ;                  (or the specific "off" pattern) to clear all pending data.
7 ;
8 ; Arguments:       None.
9 ; Return Value:    None.
10 ;
11 ; Local Variables: R16, Z (Pointer)
12 ; Shared Variables: DisplayBuffer, LightBuffer (Written)
13 ; Global Variables: None.
14 ;
15 ; Input:           None.
16 ; Output:          None.
17 ;
18 ; Error Handling:   None.
19 ; Algorithms:       None.
20 ; Data Structures:  RAM arrays for display and light states.
21 ; Registers Changed: R16, ZH, ZL
22 ;
23 ClearDisplay
24   LOAD Z pointer with address of DisplayBuffer
25   LOAD R16 with loop counter (total digits across all players)
26   ClearLoop:
27     STORE 0x00 (or "off" segment pattern) to address at Z
28     INCREMENT Z
29     DECREMENT loop counter
30     IF counter != 0 GOTO ClearLoop
31
32   LOAD Z pointer with address of LightBuffer
33   LOAD R16 with loop counter (total bytes for light matrix)
34
35   LightClearLoop:
36     STORE 0x00 to address at Z
37     INCREMENT Z
38     DECREMENT loop counter
39     IF counter != 0 GOTO LightClearLoop
40   RETURN
41
42
43 ; DisplayHex
44 ;
45 ; Description:      Outputs a 16-bit number to a specific player's 7-segment
46 ;                  display.
47 ;
48 ; Operation:       Extracts the 4 hex nibbles from the input value, looks up
```

```

48 ;           the 7-segment pattern for each in SEGTABLE, calculates the
49 ;           buffer offset for the requested player, and stores the
50 ;           patterns in RAM.
51 ;
52 ; Arguments:      n (Value to display) in R17|R16.
53 ;                 p (Player number 1-4) in R18.
54 ; Return Value:   None.
55 ;
56 ; Local Variables: R19, R20, Z (Pointer)
57 ; Shared Variables: DisplayBuffer (Written)
58 ; Global Variables: HexSegTable (in cseg)
59 ;
60 ; Input:           R17|R16 (n), R18 (p)
61 ; Output:          Updates DisplayBuffer in RAM.
62 ;
63 ; Error Handling:  If player number (p) is out of bounds (not 1-4), exit safely.
64 ; Algorithms:      Bit masking and shifting for nibble extraction.
65 ; Data Structures: RAM DisplayBuffer.
66 ; Registers Changed: R16, R17, R18, R19, R20, ZH, ZL
67 ;
68 DisplayHex:
69   CHECK if p is between 1 and 4. If not, RETURN.
70   CALCULATE starting offset in DisplayBuffer based on player p:
71     Offset = (p - 1) * 4 digits
72   LOAD Z pointer with (DisplayBuffer_Start + Offset)
73
74   FOR EACH nibble in R17|R16 (starting from most significant):
75     EXTRACT nibble by masking and shifting
76     USE nibble as index to look up pattern in HexSegTable (cseg)
77     STORE pattern to address at Z
78     INCREMENT Z pointer
79   END FOR
80   RETURN
81
82
83 ; DisplayLight
84 ;
85 ; Description:      Turns a specific pinball machine light on or off.
86 ;
87 ; Operation:        Takes a 7-bit light index, calculates the byte and bit
88 ;                   location in the LightBuffer, and sets or clears the bit
89 ;                   based on the state argument.
90 ;
91 ; Arguments:        l (Light number 0-127) in R17|R16.
92 ;                   s (State: 0=FALSE, Non-Zero=TRUE) in R18.
93 ; Return Value:     None.
94 ;
95 ; Local Variables:  R19, R20, Z (Pointer)
96 ; Shared Variables: LightBuffer (Read/Written)
97 ; Global Variables: None.

```

```

98 ;
99 ; Input:          R17|R16 (l), R18 (s)
100 ; Output:         Updates specific bit in LightBuffer in RAM.
101 ;
102 ; Error Handling:   If light number (l) is > 127, exit safely.
103 ; Algorithms:      Division/Modulo by 8 using bit shifts/masks.
104 ; Data Structures:  RAM LightBuffer.
105 ; Registers Changed: R16, R17, R18, R19, R20, ZH, ZL
106 ;
107 DisplayLight:
108   CHECK if l > 127. If so, RETURN.
109   CALCULATE byte offset: ByteOffset = l / 8 (logical shift right 3 times)
110   CALCULATE bit mask: BitIndex = l % 8 (mask lower 3 bits)
111     Create a mask with a 1 at BitIndex (e.g., 00010000)
112
113   LOAD Z pointer with (LightBuffer_Start + ByteOffset)
114   LOAD R19 with value at Z
115
116   IF s != 0 (TRUE):
117     R19 = R19 OR mask (set the bit)
118   ELSE (FALSE):
119     INVERT mask
120     R19 = R19 AND inverted_mask (clear the bit)
121
122   STORE R19 back to address at Z
123   RETURN
124
125
126 ; DisplayMuxISR
127 ;
128 ; Description:      Timer Interrupt Service Routine to multiplex displays/lights.
129 ;
130 ; Operation:        Turns off current outputs to prevent ghosting, increments the
131 ;                   multiplexing step variable, fetches the next digit/row data
132 ;                   from RAM, outputs the data, and activates the next column/digit.
133 ;
134 ; Arguments:        None.
135 ; Return Value:     None.
136 ;
137 ; Local Variables:  R16, Z (Pointer)
138 ; Shared Variables: DisplayBuffer, LightBuffer (Read), MuxStep (Read/Written)
139 ; Global Variables: PORT_SEGMENTS, PORT_DIGIT_SEL
140 ;
141 ; Input:            None (Hardware Timer Triggered)
142 ; Output:           Signals to display and digit select hardware ports.
143 ;
144 ; Error Handling:    None.
145 ; Algorithms:        None.
146 ; Data Structures:  RAM Buffers.
147 ; Registers Changed: SREG (Saved/Restored), R16, ZH, ZL

```

```
148 ;
149 DisplayMuxISR:
150     SAVE Status Register (SREG) and any used registers to Stack
151     TURN OFF all digit selections (write disable pattern to PORT_DIGIT_SEL)
152
153     LOAD R16 with current MuxStep
154     INCREMENT R16
155     IF R16 >= MAX_MUX_STEPS:
156         R16 = 0
157     STORE R16 back to MuxStep
158
159     CALCULATE data buffer address based on MuxStep
160     FETCH data pattern from DisplayBuffer/LightBuffer at that address
161     OUT data pattern to PORT_SEGMENTS / PORT_LIGHTS
162
163     FETCH digit/row selection mask based on MuxStep
164     OUT mask to PORT_DIGIT_SEL (turns on this specific digit/row)
165
166     RESTORE SREG and registers from Stack
167     RETURN FROM INTERRUPT (RETI)
```

3/hardware.asm

```
1 ; InitPorts
2 ;
3 ; Description:      Configures the I/O ports for the displays and lights.
4 ;
5 ; Operation:       Sets the data direction registers (DDR) for the segment
6 ;                  outputs, digit selection outputs, and light matrix pins to
7 ;                  output mode.
8 ;
9 ; Arguments:        None.
10 ; Return Value:    None.
11 ;
12 ; Local Variables:  R16
13 ; Shared Variables: None.
14 ; Global Variables: DDR_SEGMENTS, DDR_DIGITS, DDR_LIGHTS
15 ;
16 ; Input:            None.
17 ; Output:           None.
18 ;
19 ; Error Handling:    None.
20 ; Algorithms:        None.
21 ; Data Structures:  None.
22 ; Registers Changed: R16
23 ;
24 InitPorts:
25     LOAD R16 with 0xFF (all outputs)
26     OUT R16 to segment data direction register
27     OUT R16 to digit select data direction register
28     OUT R16 to light matrix data direction register
29     RETURN
30
31
32 ; InitDisplayTimer
33 ;
34 ; Description:      Configures the hardware timer to generate interrupts for
35 ;                  display multiplexing.
36 ;
37 ; Operation:       Sets up the timer control registers for CTC mode, sets
38 ;                  the compare value for the desired refresh rate, and
39 ;                  enables the timer interrupt.
40 ;
41 ; Arguments:        None.
42 ; Return Value:    None.
43 ;
44 ; Local Variables:  R16
45 ; Shared Variables: None.
46 ; Global Variables: TCCR_REG, TIMSK_REG, OCR_REG
47 ;
48 ; Input:            None.
```

```
49 ; Output:          None.
50 ;
51 ; Error Handling:   None.
52 ; Algorithms:      None.
53 ; Data Structures:  None.
54 ; Registers Changed: R16
55 ;
56 InitDisplayTimer:
57   LOAD R16 with timer configuration (e.g., prescaler, CTC mode)
58   OUT R16 to Timer Control Register
59   LOAD R16 with compare match value (determines multiplexing frequency)
60   OUT R16 to Output Compare Register
61   LOAD R16 with timer interrupt enable bit mask
62   OUT R16 to Timer Interrupt Mask Register
63   ENABLE global interrupts (SEI)
64   RETURN
```

3/main.asm

```
1 ; This file contains all the constants, port definitions, and register
2 ; aliases used across the project to avoid magic numbers.
3
4 DEFINE PORT_SEGMENTS as the output port for 7-segment data
5 DEFINE PORT_DIGIT_SEL as the output port for digit/player multiplexing
6 DEFINE PORT_LIGHTS as the output port for individual LED matrix rows/cols
7 DEFINE DISPLAY_BUFFER_START as the starting RAM address for display data
8 DEFINE LIGHT_BUFFER_START as the starting RAM address for light states
9
10
11 ; Main
12 ;
13 ; Description:      System entry point. Initializes hardware and starts the test
    loop.
14 ;
15 ; Operation:      Sets up the stack pointer, initializes I/O ports and timers,
16 ;                clears the display, enables interrupts, and passes control
17 ;                to the provided DisplayTest routine.
18 ;
19 ; Arguments:      None.
20 ; Return Value:   None.
21 ;
22 ; Local Variables: R16
23 ; Shared Variables: None.
24 ; Global Variables: None.
25 ;
26 ; Input:          None.
27 ; Output:         None.
28 ;
29 ; Error Handling:  None.
30 ; Algorithms:     None.
31 ; Data Structures: None.
32 ; Registers Changed: R16, SP
33 ;
34 Main:
35     INCLUDE FILES
36
37     INIT Stack Pointer to top of RAM
38     CALL InitPorts
39     CALL InitDisplayTimer
40     CALL ClearDisplay
41
42     CALL DisplayTest ; Provided in hw3test.asm, contains the loop
```